

Detecção de flaky tests utilizando LLM's

- Felipe França - faf2
- João Pedro Medeiros - jpcm2
- Lucas da Silva Carvalho - lsc8
- Pedro Vasconcelos de Menezes Casé - pvmc

Programação

1. Introdução
2. Metodologia
3. Resultados
4. Limitações

Introdução

Flaky tests são testes que **às vezes passam e às vezes falham** sem que o código tenha mudado. Eles apresentam resultados **inconsistentes**, geralmente causados por fatores como concorrência, dependências externas, tempo, ambiente ou ordem de execução.



Introdução - Impacto

16% dos 4,2 milhões de testes analisados eram flaky, causando atrasos frequentes nas releases. Além disso, entre 2% e 16% da capacidade computacional da empresa era desperdiçada apenas na reexecução desses testes instáveis (John Micco. 2017. The State of Continuous Integration Testing @Google).



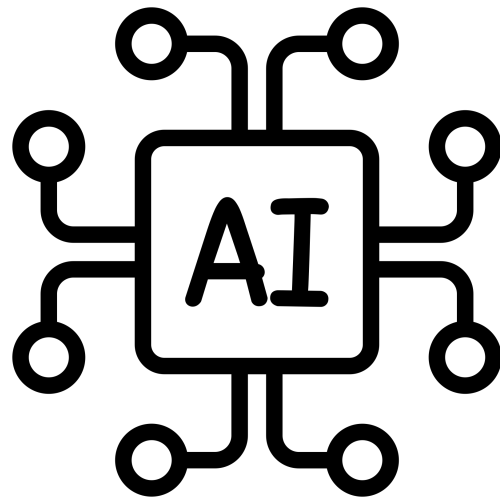
Introdução - Técnicas de identificação

- Análise estática do código para encontrar padrões suspeitos
- Análises dinâmicas com múltiplas reexecuções para detectar comportamentos inconsistentes
- Técnicas recentes também usam machine learning para prever flakiness com base em métricas de código e histórico de execuções.



Introdução - Proposta

A proposta deste trabalho é investigar a **detecção de testes flaky utilizando modelos de linguagem (LLMs)**. A abordagem busca avaliar se LLMs conseguem diferenciar testes consistentemente determinísticos daqueles que apresentam falhas intermitentes.



Programação

1. Introdução

2. Metodologia

3. Resultados

4. Limitações

Metodologia

Uso de diferentes LLMs para testar mesmos códigos e comparar resultados.

- **ChatGPT¹**
- **GeminiAI^{1,2}**
- **DeepSeek³**
- **Llama³**



1. Sem feedback
2. Com feedback
3. Execução local

Metodologia

Preparação da IA

Para deixar claro como a LLM deve responder aos prompts, elaboramos um prompt inicial explicando o objetivo da tarefa.

```
SYSTEM_PROMPT = ""
Você é um Engenheiro de QA Sênior especialista em iOS e Swift.
Sua tarefa é analisar o código de um Teste Unitário ou UI Test e classificar se ele é FLAKY (intermitente) ou NÃO.

Sinais de alerta para Flakiness:
1. Uso de `sleep()`, `Thread.sleep`, `asyncAfter` com tempos fixos.
2. Chamadas de rede reais (URLSession) sem uso de Mocks/Stubs.
3. Dependência de Data/Hora do sistema (`Date()`, `Date.now`) sem injeção.
4. Uso de números aleatórios (`Int.random`) sem semente fixa.
5. Testes de UI dependendo de animações sem expectativas (wait/expectations).
6. Race conditions (concorrência sem proteção).

IMPORTANTE:
Sempre que eu enviar um código, analise e responda APENAS um objeto JSON com este formato exato:
{
  "is_flaky": true/false,
  "confidence": "High" ou "Low",
  "reason": "Explicação técnica sucinta em 1 frase."
}
""
```

Metodologia

Prompt de análise

ChatGPT

```
# Prepara a lista de mensagens para a chamada
messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": f"Analise este código:\n\n```\n{codigo_swift}\n```"}
]

try:
    response = client.chat.completions.create(
        model=OPENAI_MODEL,
        messages=messages,
        # Força a resposta a ser JSON
        response_format={"type": "json_object"}
    )
```

GeminiAI

```
def analisar_com_chat(codigo_swift, chat_session):
    try:
        response = chat_session.send_message(f"Analise este código:\n\n```\n{codigo_swift}\n```")
        texto_limpo = response.text.replace("```json", "").replace("```", "").strip()
        return json.loads(texto_limpo)
    except Exception as e:
        return {"is_flaky": False, "error": True, "error_msg": str(e)}
```

Metodologia

Prompt de análise

Llama

```
def analisar_com_chat(codigo_swift):
    try:
        chat = {
            "messages": [
                {"role": "system", "content": SYSTEM_PROMPT},
                {
                    "role": "user",
                    "content": (
                        "Analise este código Swift e responda ESTRITAMENTE em JSON puro. "
                        "NÃO adicione explicações fora do JSON.\n\n"
                        "Código:\n```swift\n"
                        f"{codigo_swift}\n"
                        "```\n\n"
                        "Formato obrigatório:\n"
                        "{\n"
                        '  "is_flaky": true/false,\n'
                        '  "confidence": "High" ou "Low",\n'
                        '  "reason": "Explicação técnica sucinta."\n'
                        "}\n\n"
                    )
                }
            ],
            "model": "llama3.2:1b"
        }

        response = ollama.chat(
            model="llama3.2:1b",
            messages=chat["messages"]
        )
```

DeepSeek

```
def analisar_com_chat(codigo_swift, data_start):
    try:
        new_data = data_start
        new_data["prompt"] += f"\n\nAnalise este código:\n\n```swift\n{codigo_swift}\n```"

        response = requests.post(URL, json=new_data, timeout=60)
        texto_limpo = response.text.replace("```json", "").replace("```", "").strip()
        return json.loads(texto_limpo)
    except Exception as e:
        return {"is_flaky": False, "error": True, "error_msg": str(e)}
```

Metodologia

Feedback

Utilização de feedback para avaliar a melhoria nos resultados decorrente do seu uso.

Feedback Positivo: Correto! A classificação era realmente {label_correta}. O motivo '{motivo_modelo}' faz sentido. Mantenha essa lógica.

Feedback Negativo: Incorreto. Atenção: Este código ERA {label_correta}. Você disse que era o oposto por causa de '{motivo_modelo}', mas isso está errado ou insuficiente. Ajuste seus pesos para o próximo arquivo."

Metodologia - Exemplo de resposta

Acerto

[14/178]  TP (Era Flaky -> Disse Flaky)

└─ Arq: Maccy_testRemoveAndRestoreMenuBarIcon.swift

└─ Motivo: O teste utiliza UI Testing com drag and drop e expectativas baseadas em hittability.

Essas operações de UI são inerentemente sujeitas a intermitência devido a animações, tempos de resposta do sistema e outros fatores externos.

Erro

[30/178]  FN (Era Flaky -> Disse Clean)

└─ Arq:

Alamofire_testThatUploadingMultipartFormDataAboveMemoryThresholdStreamsFromDisk.swift

└─ Motivo: O teste utiliza AF.upload, mas o teste usa um limiar de 0, esperando que os dados sejam transmitidos do disco, e também verifica a ausência do arquivo, indicando que não há dependência de timing.

Metodologia

Identificação de flaky tests por análise estática com LLMs

- Utilizamos LLMs para identificar flaky tests a partir de análise estática do código.
- Cada modelo de LLM é executada separadamente no experimento, analisando individualmente os testes e classificando-os como *flaky* ou *não-flaky*.
- As justificativas fornecidas por cada modelo são utilizadas para explicar os motivos da classificação e auxiliar na interpretação das possíveis causas de flakiness.



Metodologia

Criação de um banco de dados com diferentes códigos classificados como *flaky* ou *não-flaky*:

- Análise de commits e Pull requests de repositórios open source;
- Testes executados localmente.

Aplicação de undersampling na classe majoritária.

- O banco de dados contém **91 testes flaky** e **22.354 testes não-flaky**.



Programação

1. Introdução
2. Metodologia
3. Resultados
4. Limitações

Métricas utilizadas

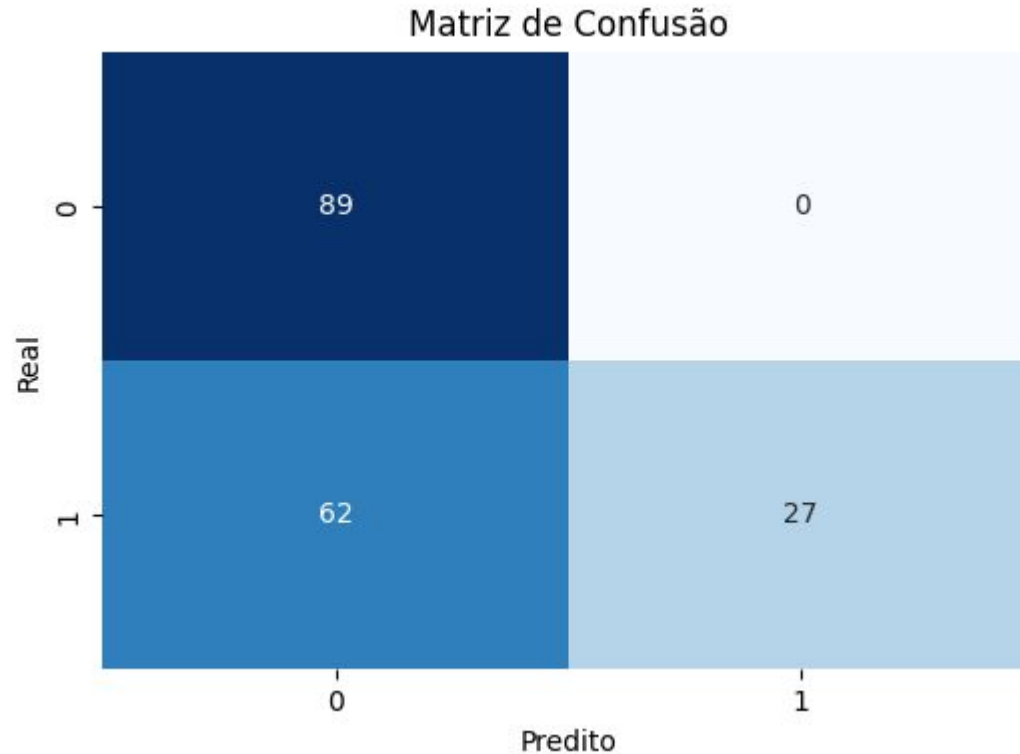
$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1 Score} = 2 \times \frac{\text{Precisão} \times \text{Recall}}{\text{Precisão} + \text{Recall}}$$

$$\text{Acurácia} = \frac{TP + TN}{TP + TN + FP + FN}$$

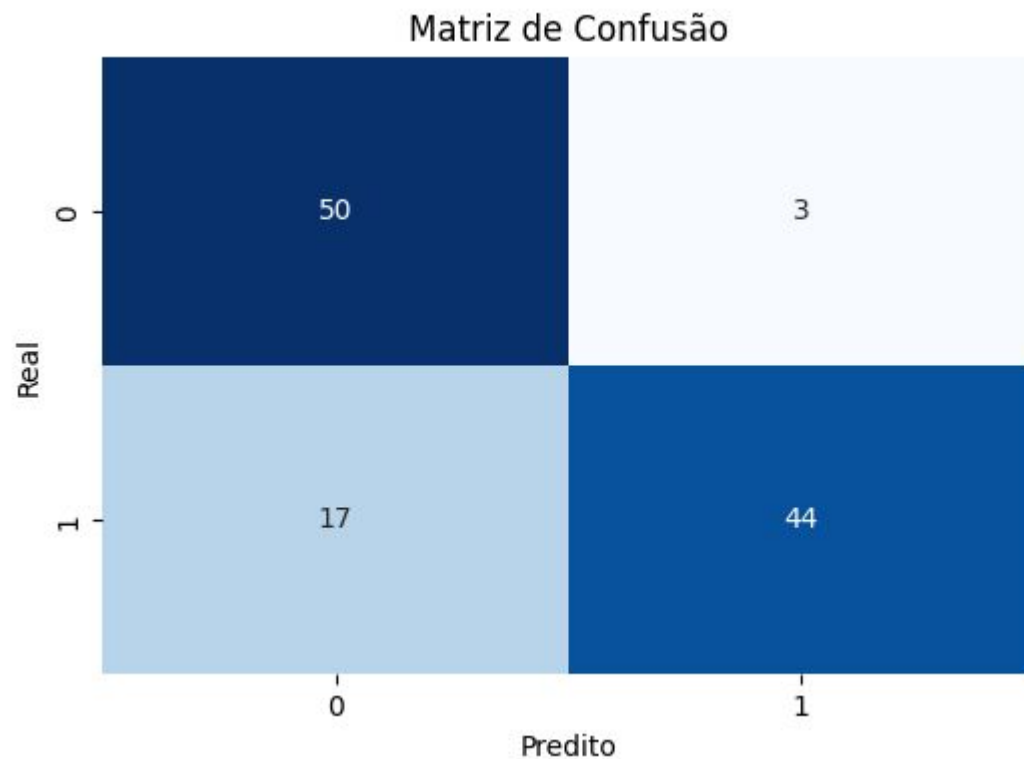
$$\text{Precisão} = \frac{TP}{TP + FP}$$

Resultados - Gemini sem feedback



 Gemini

Resultados - Gemini com feedback



 Gemini

Resultados - Gemini

Métrica	Sem Feedback	Com Feedback
---------	--------------	--------------

Acurácia	65.17%	82.46%
----------	--------	--------

F1-Score	46.55%	81.48%
----------	--------	--------

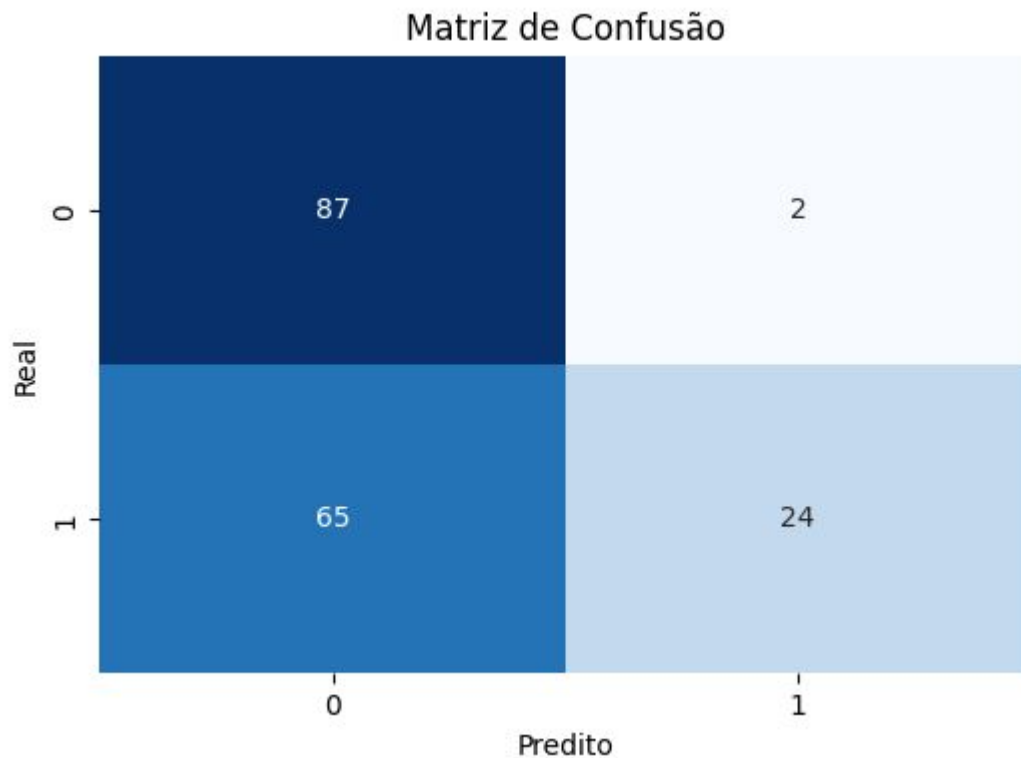
Recall	30.3%	82.4%
--------	-------	-------

Precisão	100%	93.6%
----------	------	-------



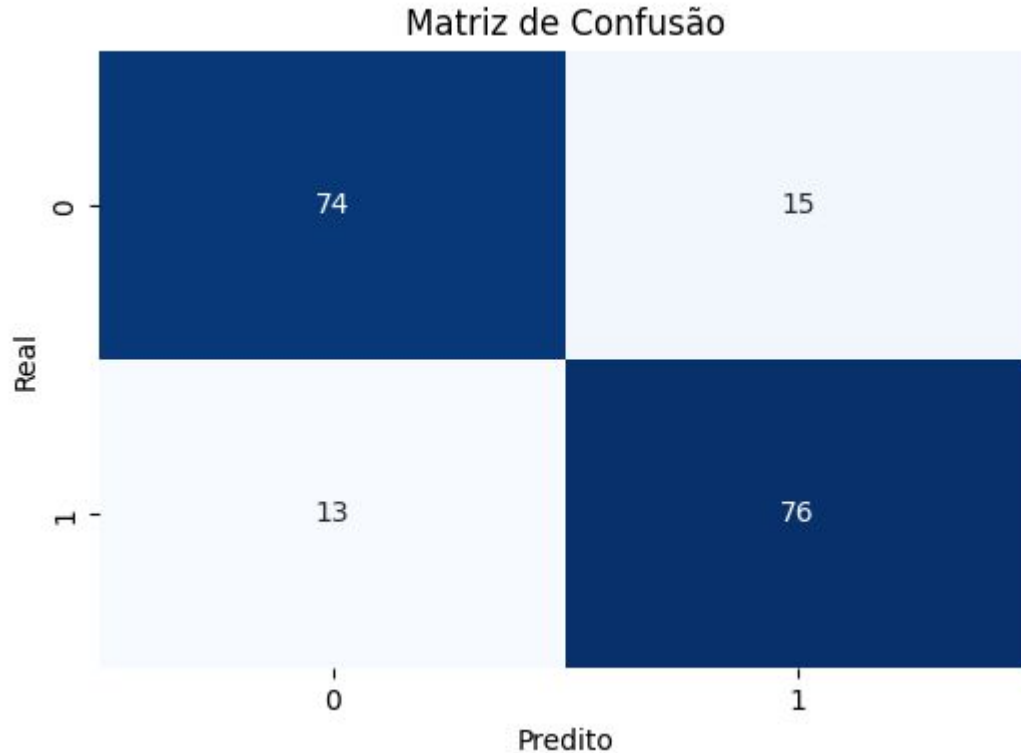
Modelo utilizado: gemini-2.5-flash-lite

Resultados - ChatGPT 3.5-Turbo sem feedback



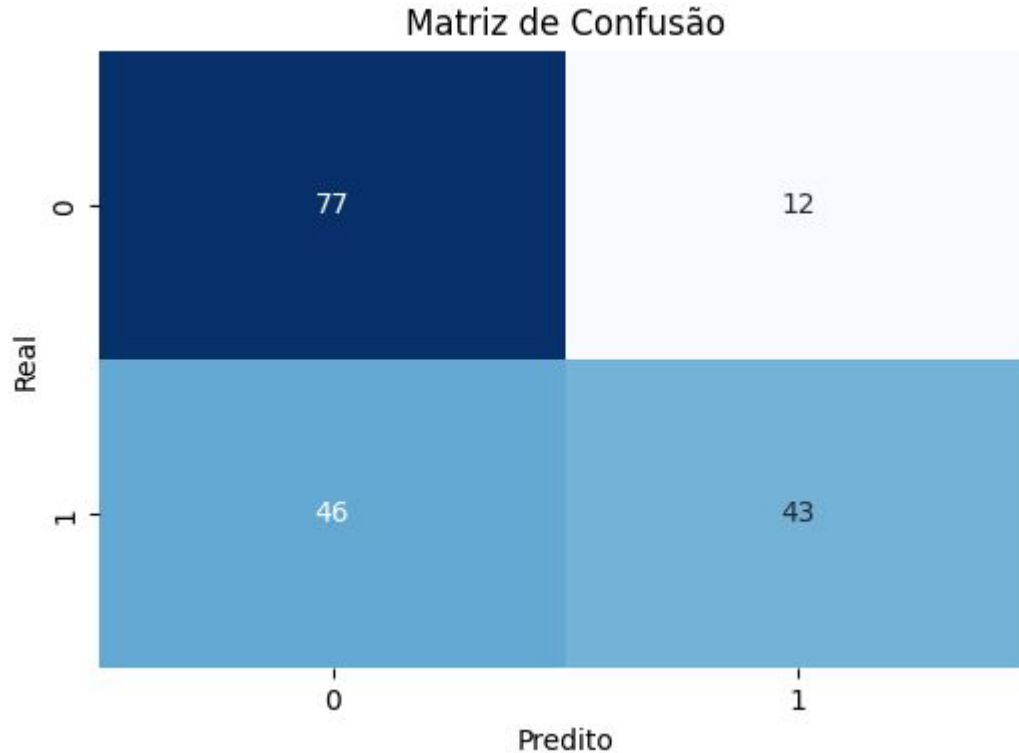
ChatGPT

Resultados - ChatGPT 4.0-Mini sem feedback



ChatGPT

Resultados - ChatGPT 4.0 sem feedback



ChatGPT

Resultados - ChatGPT sem feedback

Métrica	3.5-turbo	4.0-mini	4.0
Acurácia	62.36%	84.27%	67.42%
F1-Score	41.74%	84.44%	59.72%
Recall	26.97%	85.52%	48.31%
Precisão	92.31%	83.52%	78.18%



ChatGPT

Programação

1. Introdução
2. Metodologia
3. Resultados
4. Limitações

Limitações de LLM

1. Falta de Semântica de Execução

- LLMs fazem análise estática, mas flakiness é um fenômeno dinâmico e a LLM não tem acesso a tempo de execução.
- Ausência de contexto de Hardware/SO, latência, estado da GPU, variabilidade real.

2. Concorrência e Race Conditions

- Raciocínio sobre a ordem de execução das threads e o estado vai além do mapeamento de padrões de código (token-to-token) que os LLMs fazem bem.
- Muitos testes flaky são causados por interações indiretas entre componentes ou por bibliotecas de terceiros.

3. Limitações de Contexto

- Mesmo com janelas de contexto grandes, um LLM só consegue ver uma fração do codebase; flakiness depende de todo o código de produção que está sendo testado e as configurações do ambiente de teste.

4. Viés de Treinamento

- Bom para padrões comuns que foi indicado a destacar (e.g., `sleep()`), mas falha em identificar problemas mais sutis de flakiness.
- Risco de falsos negativos.

Limitações de LLM

5. Propensão à super confiança

- Assumem intenções do código.
- “Inventam” explicações plausíveis.
- Exageram confiança na presença/ausência de flakiness.

6. Falta de consistência entre análises

- Retorno de respostas diferentes.
- Identificar flakiness em um momento e não em outro.



Limitações de LLM

Mitigando Limitações

- Integrar LLMs a ferramentas de análise dinâmica
- Relatórios sobre o tempo de execução variável do teste.
- Rastreamento de variáveis de acesso concorrente

Isso transformaria a LLM de um classificador de texto para um raciocinador de alto nível sobre o código, atuando como um "engenheiro sênior" que recebe relatórios de ferramentas.



Limitações da pesquisa



LLMs que executadas **localmente** tiveram seu número de **parâmetros** limitado pela **RAM** disponível. Testes com **DeepSeek 1.3b** e **Llama 3.1:1b** forneceram resultados muito abaixo do esperado.

Obrigado!